

دانشگاه تهران
پردیس دانشکده‌های فنی
دانشکده مهندسی برق و کامپیوتر



یادگیری ماشین

تمرین شماره ۵

نام و نام خانوادگی
مجید صادقی نژاد

شماره دانشجویی
۸۱۰۱۰۱۴۵۹

۲۰ بهمن ۱۴۰۴

Contents

1	Question 1: Maximum Likelihood Estimation	2
2	Question 2: Logistic Regression with One Feature	3
2.1	Loss Function	3
2.2	Gradient Computation	3
2.3	SGD Update Rule	3
3	Question 3: L2 Regularized Linear Regression with Newton Method	4
3.1	Gradient and Hessian	4
3.2	Newton Update	4
3.3	Equality to the Global Minimizer	5
4	Question 4: Polynomial Regression and Ridge Regularization	5
4.1	Part A: Model Complexity Visualization	5
4.2	Part B.1: Effect of Polynomial Degree	6
4.3	Part B.2: Effect of Ridge Regularization	6
5	Question 5: Perceptron	7
5.1	1. Single Update from Point $[0, 1]$	7
5.2	2. Number of Iterations to Converge	7
5.3	3. Transformations that Lead to Convergence	8
5.4	4. One Valid Converged Weight Vector	8
5.5	5. Another Working Transformation	9
6	Question 6: Neural Network Representations	9
6.1	G1	9

6.2	G2	9
6.3	G3	10
6.4	G4	10
6.5	G5	10
6.6	G6	11
6.7	G7	11
6.8	Not Representable	11
7	Question 7: Loss Function	12
7.1	1. MLE $\Leftrightarrow$ Cross-Entropy	12
7.2	2. Noisy Labels	12
7.3	3. Targets in $\{-1, +1\}$	13
8	Question 8: Stochastic Stability and the Dead ReLU	14
8.1	1. Distribution of the Pre-Activation	14
8.2	2. Activation Probability	14
8.3	3. The 3σ Threshold	15
8.4	4. Zero-Gradient Proof	15
9	Question 9: Variance Preservation and Xavier Initialization	15
9.1	1. Symmetric Constraints	16
9.2	2. The Glorot (Xavier) Synthesis	16
9.3	3. Depth-Induced Variance	17
9.4	4. Stability Bound	17
9.5	5. Numerical Analysis	17
10	Question 10: Transfer Learning with Pre-trained Networks	18
10.1	Convergence and Stability	18
10.2	Generalization Gap	18

List of Figures

Overview

In this assignment, we explore fundamental concepts in machine learning optimization and neural networks. We begin by deriving analytical solutions for Maximum Likelihood Estimation and investigating optimization algorithms like Newton's method and SGD for classification and regression tasks. We analyze the theoretical foundations of neural networks, including their representational power, loss functions, activation stability, and weight initialization strategies. Finally, through practical implementations, we study the bias-variance trade-off in Polynomial Regression and demonstrate the effectiveness of Transfer Learning with pre-trained CNNs compared to shallow networks.

Question 1: Maximum Likelihood Estimation

Likelihood Function Given independent samples $x_1, \dots, x_n$ from

$$p(x|\theta) = \theta x^{-\theta-1}, \quad \theta > 1, x \geq 1,$$

the likelihood is

$$L(\theta) = \prod_{i=1}^n \theta x_i^{-\theta-1} = \theta^n \prod_{i=1}^n x_i^{-\theta-1}.$$

Log-Likelihood

$$\ell(\theta) = \log L(\theta) = n \log \theta - (\theta + 1) \sum_{i=1}^n \log x_i.$$

Maximization Differentiate and set to zero:

$$\frac{d\ell}{d\theta} = \frac{n}{\theta} - \sum_{i=1}^n \log x_i = 0.$$

MLE

$$\hat{\theta} = \frac{n}{\sum_{i=1}^n \log x_i}.$$

With the constraint $\theta > 1$, the MLE is

$$\hat{\theta}_{\text{MLE}} = \max\left(1, \frac{n}{\sum_{i=1}^n \log x_i}\right).$$

Question 2: Logistic Regression with One Feature

2.1 Loss Function

For a sample (x, y) with $y \in \{0, 1\}$ and

$$s = s(x - \alpha) = \frac{1}{1 + e^{-(x-\alpha)}},$$

the logistic (cross-entropy) loss is

$$J(\alpha) = -y \ln s - (1 - y) \ln(1 - s).$$

2.2 Gradient Computation

First note

$$\frac{ds}{d\alpha} = s(1 - s) \frac{d(x - \alpha)}{d\alpha} = -s(1 - s).$$

Then

$$\frac{dJ}{d\alpha} = -y \frac{1}{s} \frac{ds}{d\alpha} - (1 - y) \frac{1}{1 - s} \frac{d(1 - s)}{d\alpha}.$$

Since $\frac{d(1-s)}{d\alpha} = -\frac{ds}{d\alpha}$, we get

$$\frac{dJ}{d\alpha} = -y \frac{1}{s} \frac{ds}{d\alpha} + (1 - y) \frac{1}{1 - s} \frac{ds}{d\alpha}.$$

Substituting $\frac{ds}{d\alpha} = -s(1 - s)$:

$$\frac{dJ}{d\alpha} = y(1 - s) - s(1 - y) = y - s.$$

2.3 SGD Update Rule

With step size $\epsilon > 0$, stochastic gradient descent gives

$$\alpha \leftarrow \alpha - \epsilon \frac{dJ}{d\alpha} = \alpha - \epsilon(y - s) = \alpha + \epsilon(s - y).$$

Question 3: L2 Regularized Linear Regression with Newton Method

3.1 Gradient and Hessian

The objective is

$$J(w) = \|Xw - y\|_2^2 + \lambda\|w\|_2^2.$$

Expand:

$$J(w) = (Xw - y)^\top (Xw - y) + \lambda w^\top w.$$

The gradient is

$$\nabla J(w) = 2X^\top (Xw - y) + 2\lambda w = 2(X^\top X + \lambda I)w - 2X^\top y.$$

The Hessian is

$$\nabla^2 J(w) = 2(X^\top X + \lambda I),$$

which is constant (independent of w).

3.2 Newton Update

Newton's method updates as

$$w_1 = w_0 - [\nabla^2 J(w_0)]^{-1} \nabla J(w_0).$$

Substitute:

$$w_1 = w_0 - [2(X^\top X + \lambda I)]^{-1} (2(X^\top X + \lambda I)w_0 - 2X^\top y).$$

Factor out 2:

$$w_1 = w_0 - (X^\top X + \lambda I)^{-1} \left((X^\top X + \lambda I)w_0 - X^\top y \right).$$

3.3 Equality to the Global Minimizer

Distribute:

$$w_1 = w_0 - (X^\top X + \lambda I)^{-1} (X^\top X + \lambda I)w_0 + (X^\top X + \lambda I)^{-1} X^\top y.$$

Since

$$(X^\top X + \lambda I)^{-1} (X^\top X + \lambda I) = I,$$

we obtain

$$w_1 = w_0 - w_0 + (X^\top X + \lambda I)^{-1} X^\top y = (X^\top X + \lambda I)^{-1} X^\top y = w^*.$$

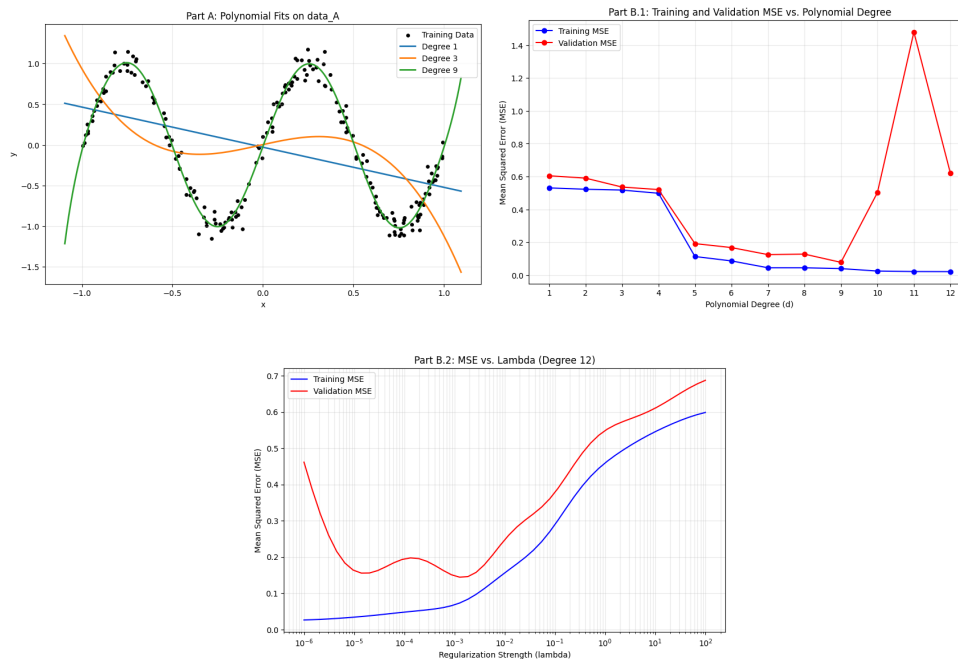
Thus one Newton step reaches the global minimizer.

Question 4: Polynomial Regression and Ridge Regularization

4.1 Part A: Model Complexity Visualization

Observations The visualization of polynomial fits for degrees $d = 1, 3, 9$ reveals distinct behaviors:

- **Degree 1 (Linear):** Clearly underfits the data, failing to capture the curvature.
- **Degree 3:** Appears to follow the trend of the data points quite well without excessive oscillations, suggesting it matches the true underlying complexity.



- **Degree 9:** Shows signs of overfitting or unnecessary complexity. It may wiggle to pass through specific noise points, which is a sign of high variance.

4.2 Part B.1: Effect of Polynomial Degree

Bias-Variance Trade-off Plotting MSE versus polynomial degree shows:

- **Underfitting:** For low degrees ($d = 1, 2$), both training and validation errors are high.
- **Overfitting:** For high degrees ($d > 7$), the training error stays low or decreases, while the validation error rises drastically.
- **Optimal Degree:** The best trade-off occurs around $d \approx 3 - 6$, where the validation error is minimized.

4.3 Part B.2: Effect of Ridge Regularization

Regularization Strength For a fixed high degree ($d = 12$), varying λ demonstrates:

- **Small λ :** The model behaves like unregularized OLS, exhibiting high variance (overfitting).
- **Large λ :** The model becomes too rigid, driving weights to zero and causing underfitting (high bias).
- **Optimum:** There exists an intermediate range for λ where validation error is minimized, balancing the fit.

Question 5: Perceptron

5.1 1. Single Update from Point $[0, 1]$

Negative class has label $y = -1$. Initial weight: $w = [0, 5]$.

$$w^\top x = [0, 5] \cdot [0, 1] = 5 > 0$$

Predicted label is $+1$, which is incorrect.

Perceptron update:

$$w \leftarrow w + yx = [0, 5] + (-1)[0, 1] = [0, 4].$$

$$w = [0, 4]$$

5.2 2. Number of Iterations to Converge

Without a bias term, the separator must pass through the origin.

On the x -axis we have:

$$(+) (-2, 0), (2, 0), \quad (-) (-1, 0), (1, 0).$$

These cannot be separated by any line through the origin since it would require

$$w_1(2) > 0 \text{ and } w_1(-2) > 0$$

simultaneously, which is impossible.

Hence the data are not linearly separable.

$$\square$$

5.3 3. Transformations that Lead to Convergence

(a) $[x, y] \rightarrow [y, x]$ Just swaps axes; separability issue remains. $\square$

(b) $[x, y] \rightarrow [x, y, 1]$ Adds bias but still cannot separate points of different radii on the same axis. $\square$

(c) $[x, y] \rightarrow [x, y, x^2, 1]$ Now ± 2 and ± 1 become separable via x^2 . $\square$

(d) $[x, y] \rightarrow [x, y, x^2 + y^2, 1]$ All values are positive; separator must pass through origin $\rightarrow$ impossible. $\square$

$$\text{Only } [x, y, x^2, 1]$$

5.4 4. One Valid Converged Weight Vector

Using transformation

$$[x, y] \rightarrow [x, y, x^2 + y^2, 1],$$

positives have $r^2 = 4$ and negatives $r^2 = 1$.

A valid separator is

$$w = [0, 0, 1, -2].$$

Check:

$$4 - 2 = 2 > 0 \quad (\text{positive}), \quad 1 - 2 = -1 < 0 \quad (\text{negative}).$$

$$w = [0, 0, 1, -2]$$

5.5 5. Another Working Transformation

Any feature that encodes radius works. One example:

$$[x, y] \rightarrow [x, y, x^2 + y^2 + x, 1].$$

Thus the blank can be:

$$x^2 + y^2 + x.$$

Question 6: Neural Network Representations

6.1 G1

Form: $f(x) = wx$ (linear through origin)

Representable:

$$(a)$$

Example parameters:

$$(a) \quad w = 2$$

6.2 G2

Form: $f(x) = wx + b$ (affine)

Representable:

$$(a), (b)$$

Example parameters:

$$(a) \ w = 2, b = 0$$

$$(b) \ w = 4, b = -5$$

6.3 G3

Form: $f(x) = \text{ReLU}(wx + b)$

Representable:

$$(c), (d)$$

Example parameters:

$$(c) \ w = 2, b = -5$$

$$(d) \ w = -2, b = -5$$

6.4 G4

Form: affine of affine $\Rightarrow$ affine

Representable:

$$(a), (b)$$

Example parameters:

$$(b) \ w_1 = 1, b_1 = 0, w_2 = 4, b_2 = -5$$

6.5 G5

Form:

$$f(x) = w_2 \text{ReLU}(w_1 x + b_1) + b_2$$

Representable:

$$(c), (d), (e)$$

Example parameters:

$$(c) \quad w_1 = 2, \quad b_1 = -5, \quad w_2 = 1, \quad b_2 = 0$$

$$(d) \quad w_1 = -2, \quad b_1 = -5, \quad w_2 = 1, \quad b_2 = 0$$

$$(e) \quad w_1 = 1, \quad b_1 = -2, \quad w_2 = -1, \quad b_2 = 1$$

6.6 G6

(Deep chain of ReLUs; multiple linear regions)

Representable:

$$(c), (d), (e), (f)$$

6.7 G7

(Two parallel ReLU units summed)

Representable:

$$(c), (d), (e), (f), (h)$$

6.8 Not Representable

$$(g) \quad \log(x)$$

Not piecewise linear $\Rightarrow$ not exactly representable by these ReLU networks.

7.1 1. MLE $\Leftrightarrow$ Cross-Entropy

For data $\{(x_n, t_n)\}_{n=1}^N$ with one-of- K targets $t_{kn} \in \{0, 1\}$ and

$$y_k(x_n, w) = p(t_k = 1 \mid x_n, w),$$

the likelihood is

$$\mathcal{L}(w) = \prod_{n=1}^N \prod_{k=1}^K y_k(x_n, w)^{t_{kn}}.$$

(Log because only the true class has $t_{kn} = 1$.)

Log-likelihood:

$$\ln \mathcal{L}(w) = \sum_{n=1}^N \sum_{k=1}^K t_{kn} \ln y_k(x_n, w).$$

Maximizing log-likelihood is equivalent to minimizing

$$E(w) = - \sum_{n=1}^N \sum_{k=1}^K t_{kn} \ln y_k(x_n, w),$$

which is the cross-entropy.

7.2 2. Noisy Labels

Let the true label be flipped with probability ϵ .

Then

$$p(t = 1 \mid x) = (1 - \epsilon)y + \epsilon(1 - y),$$

$$p(t = 0 \mid x) = (1 - \epsilon)(1 - y) + \epsilon y,$$

where $y = y(x, w)$.

Negative log-likelihood:

$$E(w) = - \sum_{n=1}^N \left[t_n \ln((1 - \epsilon)y_n + \epsilon(1 - y_n)) \right]$$

$$+(1 - t_n) \ln((1 - \epsilon)(1 - y_n) + \epsilon y_n) \Big].$$

If $\epsilon = 0$:

$$E(w) = - \sum_{n=1}^N [t_n \ln y_n + (1 - t_n) \ln(1 - y_n)],$$

which is the standard cross-entropy.

7.3 3. Targets in $\{-1, +1\}$

Let $t \in \{-1, +1\}$ and $y \in [-1, 1]$.

Map probability:

$$p(C_1 | x) = \frac{1 + y}{2}, \quad p(C_2 | x) = \frac{1 - y}{2}.$$

Unified likelihood:

$$p(t | x) = \frac{1 + ty}{2}.$$

Negative log-likelihood:

$$E(w) = - \sum_{n=1}^N \ln \frac{1 + t_n y(x_n, w)}{2}.$$

(Adding $\ln 2$ is constant, so often omitted.)

Activation function:

To ensure $y \in [-1, 1]$, use

$$\boxed{\tanh(\cdot)}$$

as the output activation.

Question 8: Stochastic Stability and the Dead ReLU

8.1 1. Distribution of the Pre-Activation

$$z = w^\top x + b = \sum_{i=1}^n w_i x_i + b.$$

Each product $w_i x_i$ has

$$\mathbb{E}[w_i x_i] = 0, \quad \text{Var}(w_i x_i) = \mathbb{E}[w_i^2] \mathbb{E}[x_i^2] = \sigma_w^2 \sigma_x^2.$$

Since terms are independent,

$$\text{Var}\left(\sum_{i=1}^n w_i x_i\right) = n \sigma_w^2 \sigma_x^2.$$

By the central limit theorem, the sum is Gaussian for large n . Thus

$$z \sim \mathcal{N}(b, \sigma_z^2), \quad \boxed{\sigma_z^2 = n \sigma_w^2 \sigma_x^2}.$$

8.2 2. Activation Probability

The neuron is active when $z > 0$.

Standardizing:

$$P(a > 0) = P(z > 0) = P\left(\frac{z - b}{\sigma_z} > \frac{-b}{\sigma_z}\right).$$

Hence

$$\boxed{P(a > 0) = \Phi\left(\frac{b}{\sigma_z}\right)},$$

where Φ is the standard normal CDF.

8.3 3. The 3σ Threshold

If

$$b = -3\sigma_z,$$

then

$$P(a > 0) = \Phi(-3) \approx 0.00135.$$

For $N = 1000$ neurons:

Expected active $\approx 1000 \times 0.00135 \approx 1.35$ (≈ 1 neuron).

8.4 4. Zero-Gradient Proof

ReLU derivative:

$$\frac{\partial a}{\partial z} = \mathbf{1}_{z>0}.$$

If $z \leq 0$ for all samples, then

$$\frac{\partial a}{\partial z} = 0$$

for every sample.

By chain rule:

$$\frac{\partial L}{\partial w} = \sum_m \frac{\partial L}{\partial a_m} \frac{\partial a_m}{\partial z_m} \frac{\partial z_m}{\partial w}.$$

But $\frac{\partial a_m}{\partial z_m} = 0$, so

$\frac{\partial L}{\partial w} = 0, \quad \Delta w = 0.$
--

Question 9: Variance Preservation and Xavier Initialization

9.1 1. Symmetric Constraints

Let a layer be

$$y_i = \sum_{j=1}^{n_{\text{in}}} W_{ij} x_j,$$

with $\mathbb{E}[W_{ij}] = 0$, $\text{Var}(W_{ij}) = \text{Var}(W)$ and $\text{Var}(x_j) = \sigma_x^2$.

Forward pass:

$$\text{Var}(y_i) = \sum_{j=1}^{n_{\text{in}}} \text{Var}(W_{ij} x_j) = n_{\text{in}} \text{Var}(W) \sigma_x^2.$$

For variance preservation $\text{Var}(y_i) = \sigma_x^2$:

$$\boxed{\text{Var}(W) = \frac{1}{n_{\text{in}}}.}$$

Backward pass:

Gradient:

$$\delta_j = \sum_{i=1}^{n_{\text{out}}} W_{ij} \delta_i^{(l+1)}.$$

Thus

$$\text{Var}(\delta_j) = n_{\text{out}} \text{Var}(W) \text{Var}(\delta^{(l+1)}).$$

Preserving gradient variance gives

$$\boxed{\text{Var}(W) = \frac{1}{n_{\text{out}}}.}$$

9.2 2. The Glorot (Xavier) Synthesis

Balance both constraints by averaging:

$$\boxed{\sigma^2 = \frac{2}{n_{\text{in}} + n_{\text{out}}}.}$$

9.3 3. Depth-Induced Variance

Let

$$G = n \operatorname{Var}(W).$$

Then one layer gives

$$\operatorname{Var}(y^{(l)}) = G \operatorname{Var}(x^{(l-1)}).$$

Recursively,

$$\operatorname{Var}(y^{(L)}) = G^L \operatorname{Var}(x^{(0)}).$$

9.4 4. Stability Bound

If

$$G = 1 + \epsilon,$$

then

$$\operatorname{Var}(y^{(L)}) = (1 + \epsilon)^L \operatorname{Var}(x^{(0)}),$$

which grows exponentially when $\epsilon > 0$.

9.5 5. Numerical Analysis

$$(1.02)^{150} \approx e^{150 \ln(1.02)} \approx e^{2.97} \approx 19.5.$$

$$\text{Amplification} \approx 19.5 \times$$

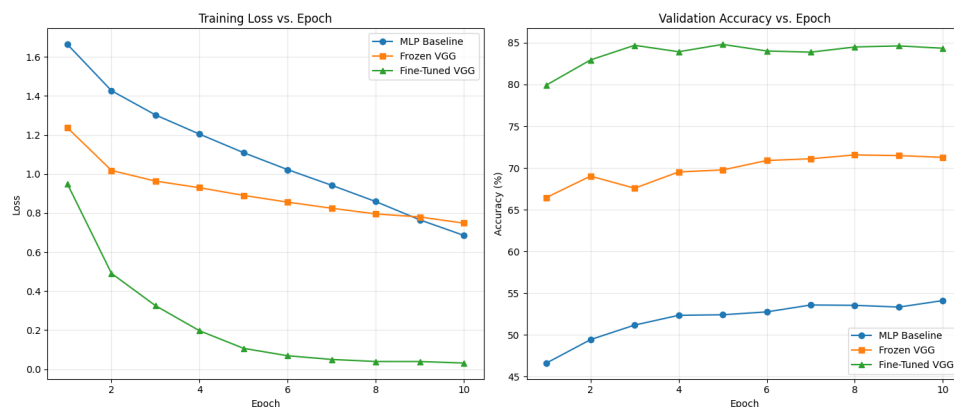
FP16 risk:

FP16 has limited dynamic range and precision. Exponential variance growth:

- pushes activations toward overflow,
- increases rounding error,
- destabilizes gradients.

Hence even small initialization errors become problematic in deep networks under FP16.

Question 10: Transfer Learning with Pre-trained Networks



10.1 Convergence and Stability

Comparison The pre-trained VGG models (both Frozen and Fine-Tuned) exhibit significantly faster convergence and smoother training curves compared to the MLP trained from scratch. The MLP often shows slower loss reduction and more fluctuations.

Reasoning Pre-trained initialization places the model parameters in a favorable region of the optimization landscape, where feature detectors (edges, textures) are already learned. Training from scratch requires learning these representations from random noise, which is a harder optimization problem susceptible to poor local minima.

10.2 Generalization Gap

Performance Even with frozen convolutional weights, the VGG network outperforms the MLP on the validation set.

Inductive Bias This gap is largely due to the spatial inductive bias of Convolutional Neural Networks (CNNs). CNNs leverage local connectivity and translation invariance to efficiently capture spatial hierarchies in images. The MLP operates on flattened vectors, discarding spatial structure, which makes generalization to unseen images much harder compared to the robust, pre-learned features of VGG.